UNIVERSITI PENDIDIKAN SULTAN IDRIS

اونيۏرسيتي ڤنديديقن سلطان ادريس

SULTAN IDRIS EDUCATION UNIVERSITY

Software Verification and Validation (DES3103)

Course Learning Outline

1. Briefing Session (Ice Breaking via Whiteboard, Class rules & Pro forma): **M. Teams**
2. Introduction to Software Verification & Validation I & II (V&V)
3. Managing V&V with SQA Integration
4. Quality metrics or attributes in V&V

WEEK 6-MID-TERM TEST (QUIZ): Topic 1 – 3-(CLO1:C2) – 20 MARKS

WEEK 7-PHASE 1: ASSIGNMENT 1-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO3:A5) - 20 MARKS: M. Teams

WEEK 9-PHASE 1: PRESENTATION & REFLECTION ON PHASE 1: M. Teams

5. Software Reviews – Will be covered on Assignments 1 & 2
6. Quality Control: Software Testing - Will be covered in Assignments 1 & 2
7. Human-Computer Interface Testing - Will be covered in Assignments 1 & 2
8. Manual Testing and Automation Testing: Will be covered in Final Assessment

WEEK 11-PHASE 2: ASSIGNMENT 2-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO2&4:P4&C5) - 20 MARKS: M. Teams
WEEK 13-PHASE 2: PRESENTATION & REFLECTION ON PHASE 2: M. Teams

9. Quality Maturity Models
10. Software Problem Analysis and Reporting
11. Quality Culture in doing V&V.
12. Safety and Security in V&V

WEEK 14: REFLECTION ON IMPLEMENTATION OPBL (SOFTWARE TESTERS) IN THE V&V COURSE: e-FORUM

WEEK 16-PHASE 3: FINAL ASSESSMENT – INDIVIDUAL SPECIAL PROJECT BASED ON PHASE 2 (CLO4:C5) (2 WEEKS) - 40 MARKS : e-ASSESSMENT

TOPIC 12

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Safety & Security in V&V

- Safety definitions, process & terminology
- Security definitions, process & terminology

Safety terminology

Term	Definition
Accident (or mishap)	An unplanned event or sequence of events which results in human death or injury, damage to property, or to the environment. An overdose of insulin is an example of an accident.
Hazard	A condition with the potential for causing or contributing to an accident. A failure of the sensor that measures blood glucose is an example of a hazard.
Damage	A measure of the loss resulting from a mishap. Damage can range from many people being killed as a result of an accident to minor injury or property damage. Damage resulting from an overdose of insulin could be serious injury or the death of the user of the insulin pump.
Hazard severity	An assessment of the worst possible damage that could result from a particular hazard. Hazard severity can range from catastrophic, where many people are killed, to minor, where only minor damage results. When an individual death is a possibility, a reasonable assessment of hazard severity is 'very high'.
Hazard probability	The probability of the events occurring which create a hazard. Probability values tend to be arbitrary but range from 'probable' (say 1/100 chance of a hazard occurring) to 'implausible' (no conceivable situations are likely in which the hazard could occur). The probability of a sensor failure in the insulin pump that results in an overdose is probably low.
Risk	This is a measure of the probability that the system will cause an accident. The risk is assessed by considering the hazard probability, the hazard severity, and the probability that the hazard will lead to an accident. The risk of an insulin overdose is probably medium to low.

Safety

- Safety is a property of a system that reflects the system's ability to operate, normally or abnormally, without danger of causing human injury or death and without damage to the system's environment.
- It is important to consider software safety as most devices whose failure is critical now incorporate software-based control systems.

Safety-critical systems

Hazards

- Situations or events that can lead to an accident
  - Stuck valve in reactor control system
  - Incorrect computation by software in navigation system
  - Failure to detect possible allergy in medication prescribing system
- Hazards do not inevitably result in accidents – accident prevention actions can be taken.

Safety achievement

- Hazard avoidance
  - The system is designed so that some classes of hazard simply cannot arise.
- Hazard detection and removal
  - The system is designed so that hazards are detected and removed before they result in an accident.
- Damage limitation
  - The system includes protection features that minimise the damage that may result from an accident.

Safety terminology

Term	Definition
Accident (or mishap)	An unplanned event or sequence of events which results in human death or injury, damage to property, or to the environment. An overdose of insulin is an example of an accident.
Hazard	A condition with the potential for causing or contributing to an accident. A failure of the sensor that measures blood glucose is an example of a hazard.
Damage	A measure of the loss resulting from a mishap. Damage can range from many people being killed as a result of an accident to minor injury or property damage. Damage resulting from an overdose of insulin could be serious injury or the death of the user of the insulin pump.
Hazard severity	An assessment of the worst possible damage that could result from a particular hazard. Hazard severity can range from catastrophic, where many people are killed, to minor, where only minor damage results. When an individual death is a possibility, a reasonable assessment of hazard severity is 'very high'.
Hazard probability	The probability of the events occurring which create a hazard. Probability values tend to be arbitrary but range from 'probable' (say 1/100 chance of a hazard occurring) to 'implausible' (no conceivable situations are likely in which the hazard could occur). The probability of a sensor failure in the insulin pump that results in an overdose is probably low.
Risk	This is a measure of the probability that the system will cause an accident. The risk is assessed by considering the hazard probability, the hazard severity, and the probability that the hazard will lead to an accident. The risk of an insulin overdose is probably medium to low.

Software safety benefits

- Although software failures can be safety-critical, the use of software control systems contributes to increased system safety
  - Software monitoring and control allows a wider range of conditions to be monitored and controlled than is possible using electro-mechanical safety systems.
  - Software control allows safety strategies to be adopted that reduce the amount of time people spend in hazardous environments.
  - Software can detect and correct safety-critical operator errors.

Safety requirements

Safety specification

- The goal of safety requirements engineering is to identify protection requirements that ensure that system failures do not cause injury or death or environmental damage.
- Safety requirements may be ‘shall not’ requirements i.e. they define situations and events that should never occur.
- Functional safety requirements define:
  - Checking and recovery features that should be included in a system
  - Features that provide protection against system failures and external attacks

Hazard-driven analysis

- Hazard identification
- Hazard assessment
- Hazard analysis
- Safety requirements specification

Hazard identification

- Identify the hazards that may threaten the system.
- Hazard identification may be based on different types of hazard:
  - Physical hazards
  - Electrical hazards
  - Biological hazards
  - Service failure hazards
  - Etc.

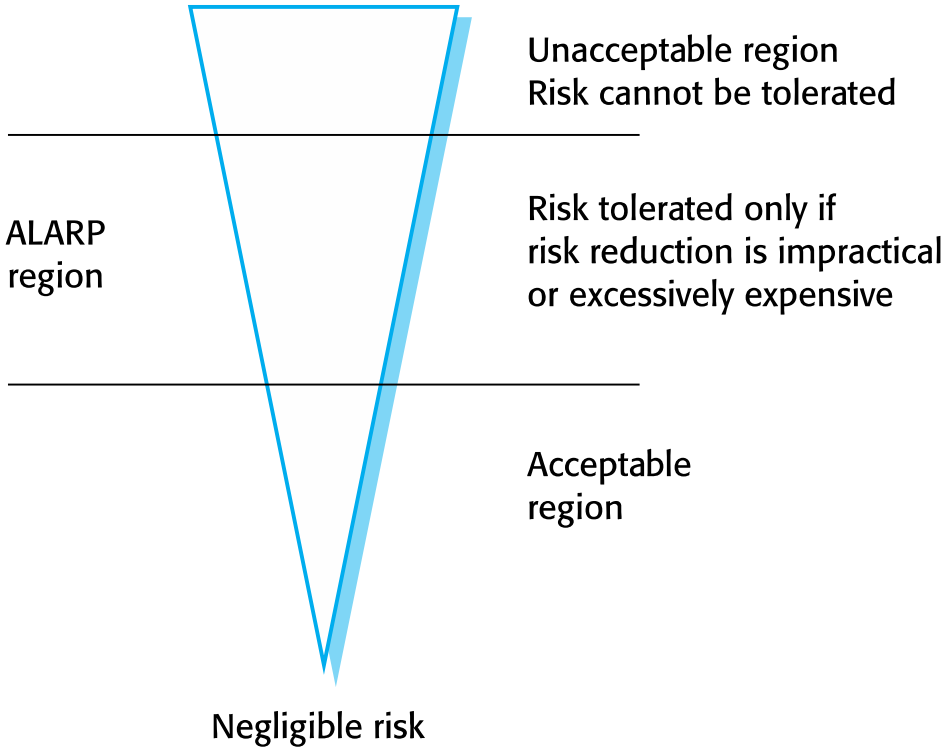
Insulin pump risks

- Insulin overdose (service failure).
- Insulin underdose (service failure).
- Power failure due to exhausted battery (electrical).
- Electrical interference with other medical equipment (electrical).
- Poor sensor and actuator contact (physical).
- Parts of machine break off in body (physical).
- Infection caused by introduction of machine (biological).
- Allergic reaction to materials or insulin (biological).

Hazard assessment

- The process is concerned with understanding the likelihood that a risk will arise and the potential consequences if an accident or incident should occur.
- Risks may be categorised as:
  - Intolerable. Must never arise or result in an accident
  - As low as reasonably practical(ALARP). Must minimise the possibility of risk given cost and schedule constraints
  - Acceptable. The consequences of the risk are acceptable and no extra costs should be incurred to reduce hazard probability

The risk triangle



Social acceptability of risk

- The acceptability of a risk is determined by human, social and political considerations.
- In most societies, the boundaries between the regions are pushed upwards with time i.e. society is less willing to accept risk
  - For example, the costs of cleaning up pollution may be less than the costs of preventing it but this may not be socially acceptable.
- Risk assessment is subjective
  - Risks are identified as probable, unlikely, etc. This depends on who is making the assessment.

Hazard assessment

- Estimate the risk probability and the risk severity.
- It is not normally possible to do this precisely so relative values are used such as ‘unlikely’, ‘rare’, ‘very high’, etc.
- The aim must be to exclude risks that are likely to arise or that have high severity.

Risk classification for the insulin pump

Identified hazard	Hazard probability	Accident severity	Estimated risk	Acceptability
1.Insulin overdose computation	Medium	High	High	Intolerable
2. Insulin underdose computation	Medium	Low	Low	Acceptable
3. Failure of hardware monitoring system	Medium	Medium	Low	ALARP
4. Power failure	High	Low	Low	Acceptable
5. Machine incorrectly fitted	High	High	High	Intolerable
6. Machine breaks in patient	Low	High	Medium	ALARP
7. Machine causes infection	Medium	Medium	Medium	ALARP
8. Electrical interference	Low	High	Medium	ALARP
9. Allergic reaction	Low	Low	Low	Acceptable

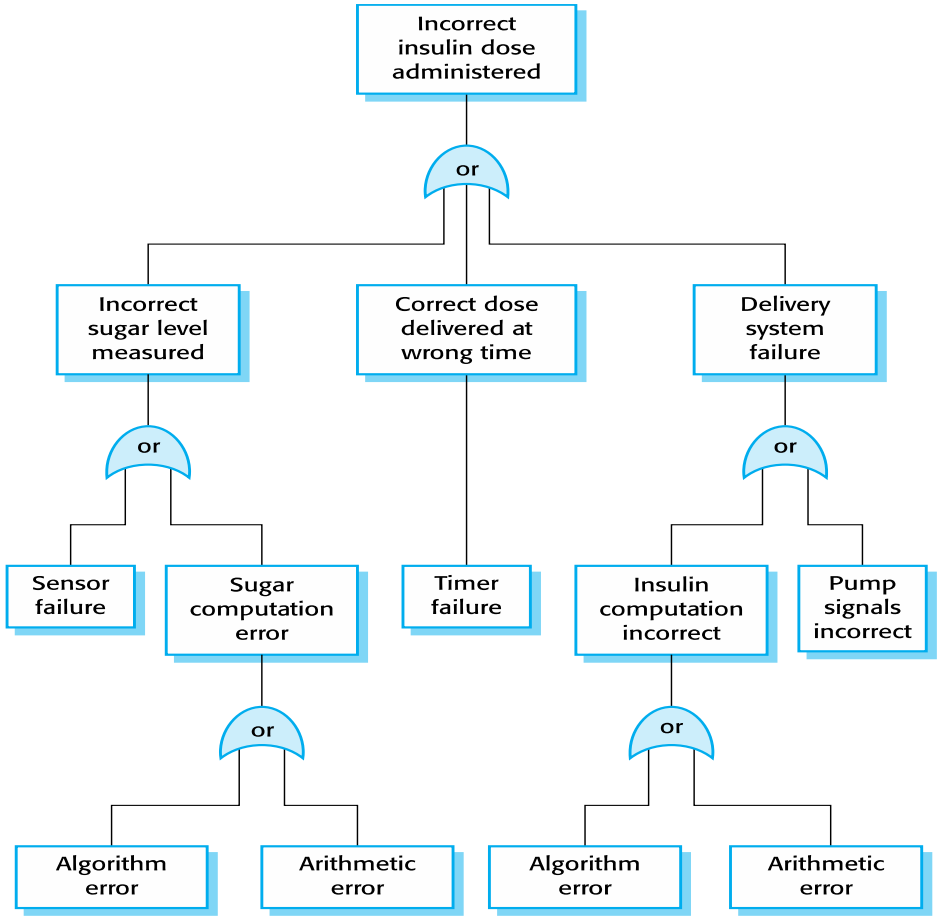
Hazard analysis

- Concerned with discovering the root causes of risks in a particular system.
- Techniques have been mostly derived from safety-critical systems and can be
  - Inductive, bottom-up techniques. Start with a proposed system failure and assess the hazards that could arise from that failure;
  - Deductive, top-down techniques. Start with a hazard and deduce what the causes of this could be.

Fault-tree analysis

- A deductive top-down technique.
- Put the risk or hazard at the root of the tree and identify the system states that could lead to that hazard.
- Where appropriate, link these with ‘and’ or ‘or’ conditions.
- A goal should be to minimise the number of single causes of system failure.

An example of a software fault tree



Fault tree analysis

- Three possible conditions that can lead to delivery of incorrect dose of insulin
  - Incorrect measurement of blood sugar level
  - Failure of delivery system
  - Dose delivered at wrong time
- By analysis of the fault tree, root causes of these hazards related to software are:
  - Algorithm error
  - Arithmetic error

Risk reduction

- The aim of this process is to identify dependability requirements that specify how the risks should be managed and ensure that accidents/incidents do not arise.
- Risk reduction strategies
  - Hazard avoidance;
  - Hazard detection and removal;
  - Damage limitation.

Strategy use

- Normally, in critical systems, a mix of risk reduction strategies are used.
- In a chemical plant control system, the system will include sensors to detect and correct excess pressure in the reactor.
- However, it will also include an independent protection system that opens a relief valve if dangerously high pressure is detected.

Insulin pump - software risks

- Arithmetic error
  - A computation causes the value of a variable to overflow or underflow;
  - Maybe include an exception handler for each type of arithmetic error.
- Algorithmic error
  - Compare dose to be delivered with previous dose or safe maximum doses. Reduce dose if too high.

Examples of safety requirements

SR1: The system shall not deliver a single dose of insulin that is greater than a specified maximum dose for a system user.

SR2: The system shall not deliver a daily cumulative dose of insulin that is greater than a specified maximum daily dose for a system user.

SR3: The system shall include a hardware diagnostic facility that shall be executed at least four times per hour.

SR4: The system shall include an exception handler for all of the exceptions that are identified in Table 3.

SR5: The audible alarm shall be sounded when any hardware or software anomaly is discovered and a diagnostic message, as defined in Table 4, shall be displayed.

SR6: In the event of an alarm, insulin delivery shall be suspended until the user has reset the system and cleared the alarm.

Aspek yang perlu diambilkira dalam menjamin keselamatan sistem adalah:

- Aspek yang perlu diambilkira dalam menjamin keselamatan sistem adalah:

a) Prinsip umum Pembangunan Sistem Terselamat

Keselamatan sistem boleh diancam pada mana-mana aktiviti sepanjang kitaran hayatnya, sama ada secara sengaja atau tidak sengaja oleh "orang dalam" atau oleh "orang luar" yang tidak mempunyai hubungan dengan organisasi. Penambahbaikan kelemahan yang dikesan pada peringkat awal sepanjang kitar hayat sistem aplikasi adalah lebih kos efektif daripada membangun dan mengeluarkan versi/patch keselamatan baru untuk sistem beroperasi. Untuk dianggap selamat, sistem mestilah mempunyai ciri-ciri:

i) Kebergantungan (*Dependability)*

Sistem yang dibangunkan boleh beroperasi dengan lancar di bawah semua keadaan dan platform, termasuklah sekiranya terdapat serangan atau pelbagai niat jahat.

ii) Dipercayai *(Trustworthiness)*

Sistem boleh dipercayai walaupun terdapat kelemahan yang sengaja dieksploitasi untuk mengganggu pengoperasian sistem. Sistem yang boleh dipercayai mesti mengandungi logik yang boleh menghalang sebarang pencerobohan jahat.

iii) Daya Tahan (*Resilience*)

Sistem yang berdaya tahan adalah sistem yang mampu menentang serangan atau dapat dipulihkan dengan cepat sekiranya ada serangan yang tidak boleh ditolak/ ditahan.

7 Ciri-Ciri Keselamatan Utama Pembangunan Sistem

i) Keselamatan Tahap Sistem

- Sistem dapat mengawal akses aplikasi pada peranan setiap pengguna. Biasanya sistem dikawal berasaskan paparan pilihan menu yang berbeza mengikut peranan pengguna.

ii) Keselamatan *Row-Level*
Pengguna hanya dapat melihat data yang dibenarkan mengikut peranan

- mereka di dalam aplikasi yang sama.

iii) *SingleSign-On*(SSO)

- Satu proses pengesahan pengguna yang membolehkan pengguna memasukkan id penggguna dan kata laluan mereka pada satu sistem sahaja tetapi pengesahan tersebut memberi kebenaran kepada pengguna untuk mengakses sistem lain yang berkaitan pada sesi yang sama tanpa perlu log masuk ke sistem tersebut. SSO akan mengurangkan bilangan ID Pengguna/ kata laluan yang perlu diingat dan memerlukan login pada setiap aplikasi yang diperlukan.

7 Ciri-Ciri Keselamatan Utama Pembangunan Sistem

iv) Parameter Keistimewaaan Pengguna

- Parameter keistimewaan pengguna digunakan untuk memperibadikan ciri dan keselamatan kepada pengguna individu atau peranan pengguna. Parameter keistimewaan pengguna disimpan ke profil pengguna dan boleh diakses pada keseluruhan sistem. Ianya sangat fleksibel, dapat mengawal, menambah atau menyembunyi pilihan pengguna. Contoh: walaupun semua pengguna boleh mengakses aplikasi yang sama, tetapi hanya pengguna tertentu sahaja yang boleh melihat pilihan untuk mengemaskini data.

v) Pilihan Pengesahan Fleksibel (*Flexible Authentication Options*)

- Sistem aplikasi yang dibangunkan sepatutnya menawarkan pelbagai pilihan kaedah pengesahan pengguna, sistem sepatutnya fleksibel untuk menggunakan kaedah pengesahan yang sedia ada tanpa perlu wujud atau mengubah kaedah pengesahan semasa.

7 Ciri-Ciri Keselamatan Utama Pembangunan Sistem

vi) Sumber Data Pengguna (*User Specific Data Source*)

- Ciri keselamatan ini adalah sama dengan row-level keselamatan, tetapi di peringkat pangkalan data. Aplikasi yang dibangunkan boleh mengakses sumber data yang berbeza bergantung kepada pengguna. Contoh: Apabila berlaku penggabungan 2 syarikat yang mengguna sistem yang sama. Syarikat A perlu akses pangkalan data *local*, manakala pekerja syarikat B perlu akses data dari pangkalan data berlainan.

vii) Pengauditan Aktiviti Sistem

- Pengauditan aktiviti sistem membolehkan pembangun log aktiviti pengguna akhir untuk setiap aktiviti *Sign On / Sign Off*. Ini membolehkan sistem menjejaki dengan cepat dan merekod log masuk dan keluar pengguna, fungsi yang dicapai dan aktiviti yang dilakukan terhadap data.

10 senarai teratas ancaman yang disenaraikan *Open Web Application Security Project* (*OWASP*) 2017 seperti berikut :

1. *SQL Injection* – data yang tidak sah disuntik ke dalam sistem untuk membuka ruang kepada capaian data di dalam pangkalan data sistem. Contoh ancaman: *SQL, OS* dan *LDAP injection*.
2. *Broken Authentication and Session Management* – kelemahan fungsi aplikasi berkaitan dengan proses pengesahan (*authentication*) dan pengurusan *session* membolehkan penyerang memanipulasi kata kunci dan token *session* untuk memanipulasi akses pengguna sedia ada.
3. iii) *Sensitive Data Exposure* – sistem aplikasi hendaklah melindungi data sensitif seperti maklumat kad kredit, nombor akaun pengguna, nombor ID pengguna (seperti nombor passport, nombor kad pengenalan) serta maklumat akses pengguna agar tidak dimanipulasi oleh pihak tidak bertanggungjawab.
4. iv) *XML External Entities (XXE)* – Kelemahan konfigurasi ke atas pemproses XML membuka ruang kepada serangan seperti serangan *denial of service* (DOS).
5. *Broken Access Control* – kelemahan kawalan dan penguatkuasaan terhadap aktiviti yang dibenarkan ke atas pengguna yang sah membuka ruang kepada capaian maklumat yang sulit dan terperingkat.

10 senarai teratas ancaman yang disenaraikan *Open Web Application Security Project* (*OWASP*) 2017 seperti berikut :

6. *Security Misconfiguration* – sesuatu sistem yang baik perlu dikonfigurasi di peringkat aplikasi, kerangka, pelayan aplikasi, pelayan web, pelayan pangkalan data dan platform serta tidak menggunakan konfigurasi *default* semata-mata. Selain itu, perisian juga perlu sentiasa dikemaskini.
7. *Cross-Site Scripting (XSS)* – penyerang akan melaksanakan skrip di dalam browser mangsa dan seterusnya berupaya memanipulasi *user sessions*, mengubah laman web sedia ada atau memesongkan pengguna kepada laman web lain.
8. *Insecure Deserialization* - penyerang berupaya membuat serangan bersiri ke atas kod aturcara secara *remote.*
9. *Using Components with Known Vulnerabilities* – eksploitasi ke atas komponen seperti *libraries*, kerangka dan modul perisian boleh menyebabkan kehilangan data yang serius atau manipulasi ke atas pelayan.
10. *Insufficient Logging and Monitoring* – kelemahan kawalan ke atas log masuk sistem membolehkan penyerang memecah masuk dan merosakkan data di dalam sistem.

Safety engineering processes

Safety engineering processes

- Safety engineering processes are based on reliability engineering processes
  - Plan-based approach with reviews and checks at each stage in the process
  - General goal of fault avoidance and fault detection
  - Must also include safety reviews and explicit identification and tracking of hazards

Regulation

- Regulators may require evidence that safety engineering processes have been used in system development
- For example:
  - The specification of the system that has been developed and records of the checks made on that specification.
  - Evidence of the verification and validation processes that have been carried out and the results of the system verification and validation.
  - Evidence that the organizations developing the system have defined and dependable software processes that include safety assurance reviews. There must also be records that show that these processes have been properly enacted.

Agile methods and safety

- Agile methods are not usually used for safety-critical systems engineering
  - Extensive process and product documentation is needed for system regulation. Contradicts the focus in agile methods on the software itself.
  - A detailed safety analysis of a complete system specification is important. Contradicts the interleaved development of a system specification and program.
- Some agile techniques such as test-driven development may be used

Safety assurance processes

- Process assurance involves defining a dependable process and ensuring that this process is followed during the system development.
- Process assurance focuses on:
  - Do we have the right processes? Are the processes appropriate for the level of dependability required. Should include requirements management, change management, reviews and inspections, etc.
  - Are we doing the processes right? Have these processes been followed by the development team.
- Process assurance generates documentation
  - Agile processes therefore are rarely used for critical systems.

Processes for safety assurance

- Process assurance is important for safety-critical systems development:
  - Accidents are rare events so testing may not find all problems;
  - Safety requirements are sometimes ‘shall not’ requirements so cannot be demonstrated through testing.
- Safety assurance activities may be included in the software process that records the analyses that have been carried out and the people responsible for these.
  - Personal responsibility is important as system failures may lead to subsequent legal actions.

Safety related process activities

- Creation of a hazard logging and monitoring system.
- Appointment of project safety engineers who have explicit responsibility for system safety.
- Extensive use of safety reviews.
- Creation of a safety certification system where the safety of critical components is formally certified.
- Detailed configuration management (see Chapter 25).

Hazard analysis

- Hazard analysis involves identifying hazards and their root causes.
- There should be clear traceability from identified hazards through their analysis to the actions taken during the process to ensure that these hazards have been covered.
- A hazard log may be used to track hazards throughout the process.

A simplified hazard log entry

Hazard Log	**Page 4: Printed 20.02.2012**				
System: Insulin Pump System *Safety Engineer:* James Brown				*File*: InsulinPump/Safety/HazardLog *Log version*: 1/3	
Identified Hazard	Insulin overdose delivered to patient				
Identified by	Jane Williams				
Criticality class	1				
Identified risk	High				
Fault tree identified	YES	*Date*	24.01.07	*Location*	Hazard Log, Page 5
Fault tree creators	Jane Williams and Bill Smith				
Fault tree checked	YES	*Date*	28.01.07	*Checker*	James Brown

Hazard log (2)

System safety design requirements
1. The system shall include self-testing software that will test the sensor system, the clock, and the insulin delivery system.
2. The self-checking software shall be executed once per minute.
3. In the event of the self-checking software discovering a fault in any of the system components, an audible warning shall be issued and the pump display shall indicate the name of the component where the fault has been discovered. The delivery of insulin shall be suspended.
4. The system shall incorporate an override system that allows the system user to modify the computed dose of insulin that is to be delivered by the system.
5. The amount of override shall be no greater than a pre-set value (maxOverride), which is set when the system is configured by medical staff.

Safety reviews

- Driven by the hazard register.
- For each identified hazard, the review team should assess the system and judge whether or not the system can cope with that hazard in a safe way.

Automated static analysis checks

Fault class	Static analysis check
Data faults	Variables used before initialization Variables declared but never used Variables assigned twice but never used between assignments Possible array bound violations Undeclared variables
Control faults	Unreachable code Unconditional branches into loops
Input/output faults	Variables output twice with no intervening assignment
Interface faults	Parameter-type mismatches Parameter number mismatches Non-usage of the results of functions Uncalled functions and procedures
Storage management faults	Unassigned pointers Pointer arithmetic Memory leaks

Safety cases

Safety and dependability cases

- Safety and dependability cases are structured documents that set out detailed arguments and evidence that a required level of safety or dependability has been achieved.
- They are normally required by regulators before a system can be certified for operational use. The regulator's responsibility is to check that a system is as safe or dependable as is practical.
- Regulators and developers work together and negotiate what needs to be included in a system safety/dependability case.

The system safety case

- A safety case is:
  - A documented body of evidence that provides a convincing and valid argument that a system is adequately safe for a given application in a given environment.
- Arguments in a safety case can be based on formal proof, design rationale, safety proofs, etc. Process factors may also be included.
- A software safety case is usually part of a wider system safety case that takes hardware and operational issues into account.

The contents of a software safety case

Chapter	Description
System description	An overview of the system and a description of its critical components.
Safety requirements	The safety requirements abstracted from the system requirements specification. Details of other relevant system requirements may also be included.
Hazard and risk analysis	Documents describing the hazards and risks that have been identified and the measures taken to reduce risk. Hazard analyses and hazard logs.
Design analysis	A set of structured arguments (see Section 15.5.1) that justify why the design is safe.
Verification and validation	A description of the V & V procedures used and, where appropriate, the test plans for the system. Summaries of the test results showing defects that have been detected and corrected. If formal methods have been used, a formal system specification and any analyses of that specification. Records of static analyses of the source code.

Chapter	Description
Review reports	Records of all design and safety reviews.
Team competences	Evidence of the competence of all of the team involved in safety-related systems development and validation.
Process QA	Records of the quality assurance processes (see Chapter 24) carried out during system development.
Change management processes	Records of all changes proposed, actions taken and, where appropriate, justification of the safety of these changes. Information about configuration management procedures and configuration management logs.
Associated safety cases	References to other safety cases that may impact the safety case.

Key points

- Safety-critical systems are systems whose failure can lead to human injury or death.
- A hazard-driven approach is used to understand the safety requirements for safety-critical systems. You identify potential hazards and decompose these (using methods such as fault tree analysis) to discover their root causes. You then specify requirements to avoid or recover from these problems.
- It is important to have a well-defined, certified process for safety-critical systems development. This should include the identification and monitoring of potential hazards.

Key points

- Static analysis is an approach to V & V that examines the source code of a system, looking for errors and anomalies. It allows all parts of a program to be checked, not just those parts that are exercised by system tests.
- Model checking is a formal approach to static analysis that exhaustively checks all states in a system for potential errors.
- Safety and dependability cases collect the evidence that demonstrates a system is safe and dependable. Safety cases are required when an external regulator must certify the system before it is used.

Security engineering

- Tools, techniques and methods to support the development and maintenance of systems that can resist malicious attacks that are intended to damage a computer-based system or its data.
- A sub-field of the broader field of computer security.

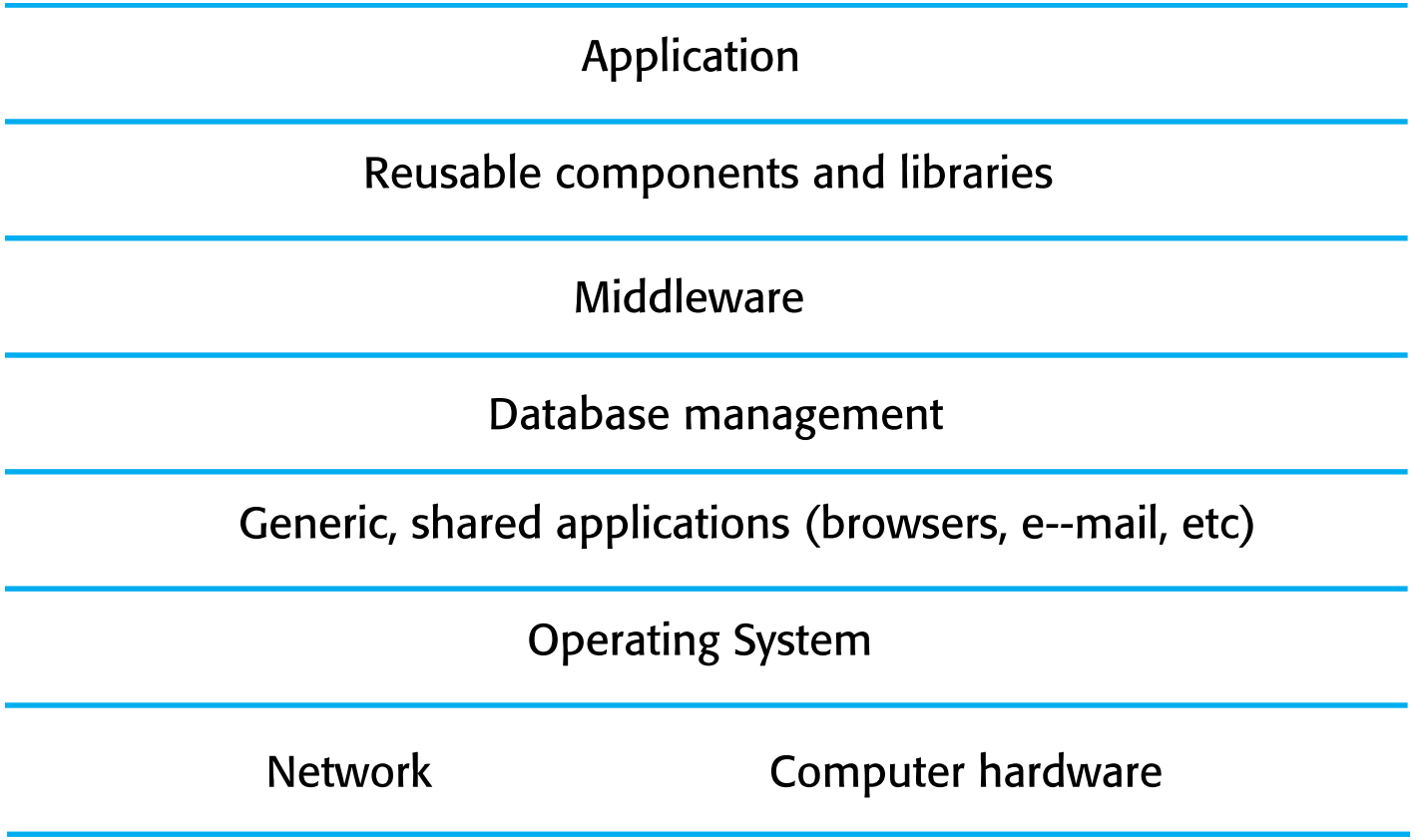
Security dimensions

- *Confidentiality*
  - Information in a system may be disclosed or made accessible to people or programs that are not authorized to have access to that information.
- *Integrity*
  - Information in a system may be damaged or corrupted making it unusual or unreliable.
- *Availability*
  - Access to a system or its data that is normally available may not be possible.

Security levels

- Infrastructure security, which is concerned with maintaining the security of all systems and networks that provide an infrastructure and a set of shared services to the organization.
- Application security, which is concerned with the security of individual application systems or related groups of systems.
- Operational security, which is concerned with the secure operation and use of the organization’s systems.

System layers where security may be compromised



Application/infrastructure security

- Application security is a software engineering problem where the system is designed to resist attacks.
- Infrastructure security is a systems management problem where the infrastructure is configured to resist attacks.
- The focus of this chapter is application security rather than infrastructure security.

System security management

- User and permission management
  - Adding and removing users from the system and setting up appropriate permissions for users
- Software deployment and maintenance
  - Installing application software and middleware and configuring these systems so that vulnerabilities are avoided.
- Attack monitoring, detection and recovery
  - Monitoring the system for unauthorized access, design strategies for resisting attacks and develop backup and recovery strategies.

Operational security

- Primarily a human and social issue
- Concerned with ensuring the people do not take actions that may compromise system security
  - E.g. Tell others passwords, leave computers logged on
- Users sometimes take insecure actions to make it easier for them to do their jobs
- There is therefore a trade-off between system security and system effectiveness.

Security and dependability

Security

- The security of a system is a system property that reflects the system's ability to protect itself from accidental or deliberate external attack.
- Security is essential as most systems are networked so that external access to the system through the Internet is possible.
- Security is an essential pre-requisite for availability, reliability and safety.

Fundamental security

- If a system is a networked system and is insecure then statements about its reliability and its safety are unreliable.
- These statements depend on the executing system and the developed system being the same. However, intrusion can change the executing system and/or its data.
- Therefore, the reliability and safety assurance is no longer valid.

Security terminology

Term	Definition
Asset	Something of value which has to be protected. The asset may be the software system itself or data used by that system.
Attack	An exploitation of a system's vulnerability. Generally, this is from outside the system and is a deliberate attempt to cause some damage.
Control	A protective measure that reduces a system's vulnerability. Encryption is an example of a control that reduces a vulnerability of a weak access control system
Exposure	Possible loss or harm to a computing system. This can be loss or damage to data, or can be a loss of time and effort if recovery is necessary after a security breach.
Threat	Circumstances that have potential to cause loss or harm. You can think of these as a system vulnerability that is subjected to an attack.
Vulnerability	A weakness in a computer-based system that may be exploited to cause loss or harm.

Examples of security terminology (Mentcare)

Term	Example
Asset	The records of each patient that is receiving or has received treatment.
Exposure	Potential financial loss from future patients who do not seek treatment because they do not trust the clinic to maintain their data. Financial loss from legal action by the sports star. Loss of reputation.
Vulnerability	A weak password system which makes it easy for users to set guessable passwords. User ids that are the same as names.
Attack	An impersonation of an authorized user.
Threat	An unauthorized user will gain access to the system by guessing the credentials (login name and password) of an authorized user.
Control	A password checking system that disallows user passwords that are proper names or words that are normally included in a dictionary.

Security and organizations

Security policies

- *The assets that must be protected*
  - It is not cost-effective to apply stringent security procedures to all organizational assets. Many assets are not confidential and can be made freely available.
- *The level of protection that is required for different types of asset*
  - For sensitive personal information, a high level of security is required; for other information, the consequences of loss may be minor so a lower level of security is adequate.

Security policies

- *The responsibilities of individual users, managers and the organization*
  - The security policy should set out what is expected of users e.g. strong passwords, log out of computers, office security, etc.
- *Existing security procedures and technologies that should be maintained*
  - For reasons of practicality and cost, it may be essential to continue to use existing approaches to security even where these have known limitations.

Security requirements

Security specification

- Security specification has something in common with safety requirements specification – in both cases, your concern is to avoid something bad happening.
- Four major differences
  - Safety problems are accidental – the software is not operating in a hostile environment. In security, you must assume that attackers have knowledge of system weaknesses
  - When safety failures occur, you can look for the root cause or weakness that led to the failure. When failure results from a deliberate attack, the attacker may conceal the cause of the failure.
  - Shutting down a system can avoid a safety-related failure. Causing a shut down may be the aim of an attack.
  - Safety-related events are not generated from an intelligent adversary. An attacker can probe defenses over time to discover weaknesses.

Asset analysis in a preliminary risk assessment report for the Mentcare system

Asset	Value	Exposure
The information system	High. Required to support all clinical consultations. Potentially safety-critical.	High. Financial loss as clinics may have to be canceled. Costs of restoring system. Possible patient harm if treatment cannot be prescribed.
The patient database	High. Required to support all clinical consultations. Potentially safety-critical.	High. Financial loss as clinics may have to be canceled. Costs of restoring system. Possible patient harm if treatment cannot be prescribed.
An individual patient record	Normally low although may be high for specific high-profile patients.	Low direct losses but possible loss of reputation.

Threat and control analysis in a preliminary risk assessment report

Threat	Probability	Control	Feasibility
An unauthorized user gains access as system manager and makes system unavailable	Low	Only allow system management from specific locations that are physically secure.	Low cost of implementation but care must be taken with key distribution and to ensure that keys are available in the event of an emergency.
An unauthorized user gains access as system user and accesses confidential information	High	Require all users to authenticate themselves using a biometric mechanism. Log all changes to patient information to track system usage.	Technically feasible but high-cost solution. Possible user resistance. Simple and transparent to implement and also supports recovery.

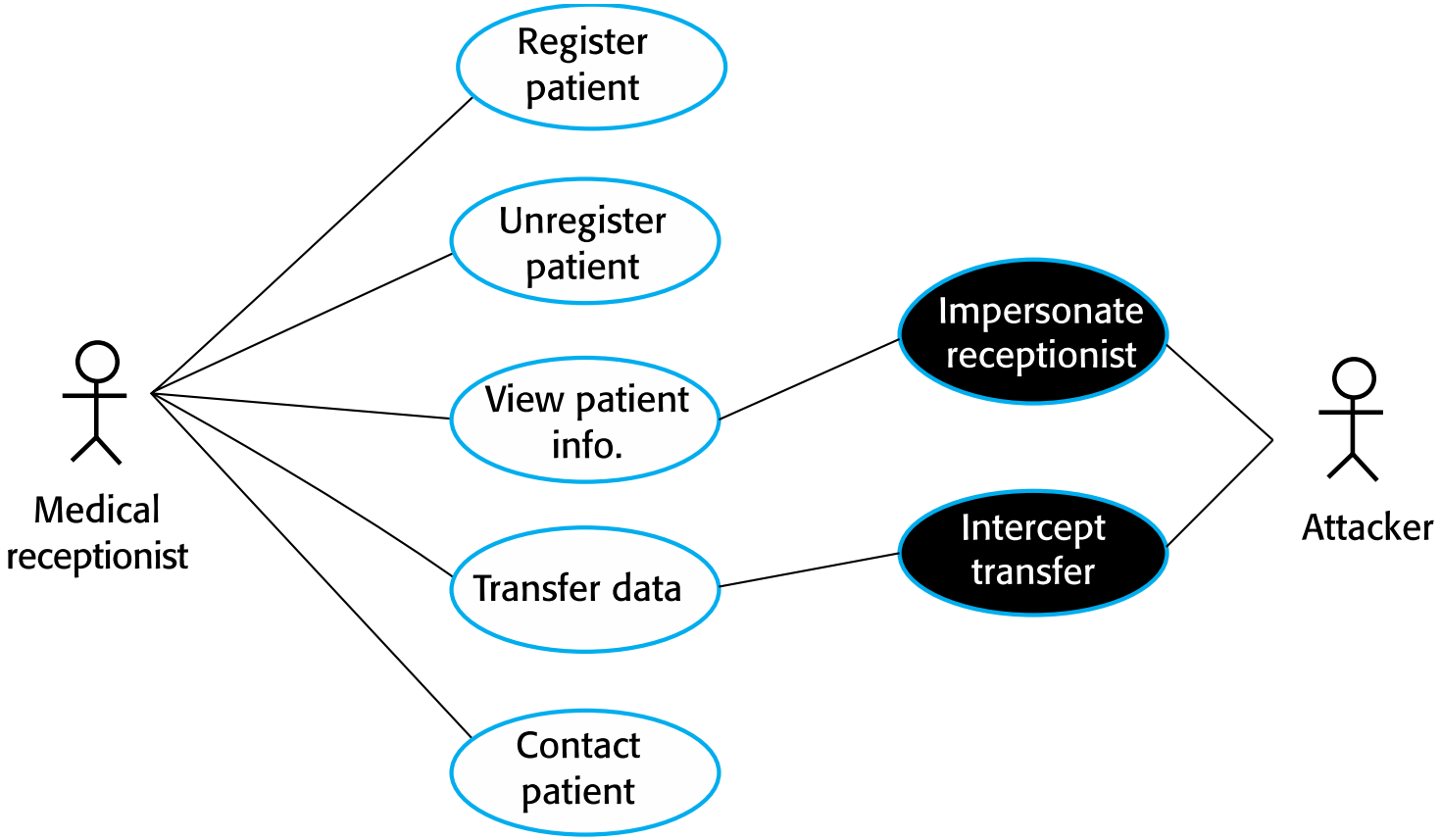
Security requirements for the Mentcare system

- Patient information shall be downloaded at the start of a clinic session to a secure area on the system client that is used by clinical staff.
- All patient information on the system client shall be encrypted.
- Patient information shall be uploaded to the database after a clinic session has finished and deleted from the client computer.
- A log on a separate computer from the database server must be maintained of all changes made to the system database.

Misuse cases

- Misuse cases are instances of threats to a system
- Interception threats
  - Attacker gains access to an asset
- Interruption threats
  - Attacker makes part of a system unavailable
- Modification threats
  - A system asset if tampered with
- Fabrication threats
  - False information is added to a system

Misuse cases



Mentcare use case – Transfer data

Mentcare system: Transfer data	
Actors	Medical receptionist, Patient records system (PRS)
Description	A receptionist may transfer data from the Mentcare system to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary.
Stimulus	User command issued by medical receptionist.
Response	Confirmation that PRS has been updated.
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

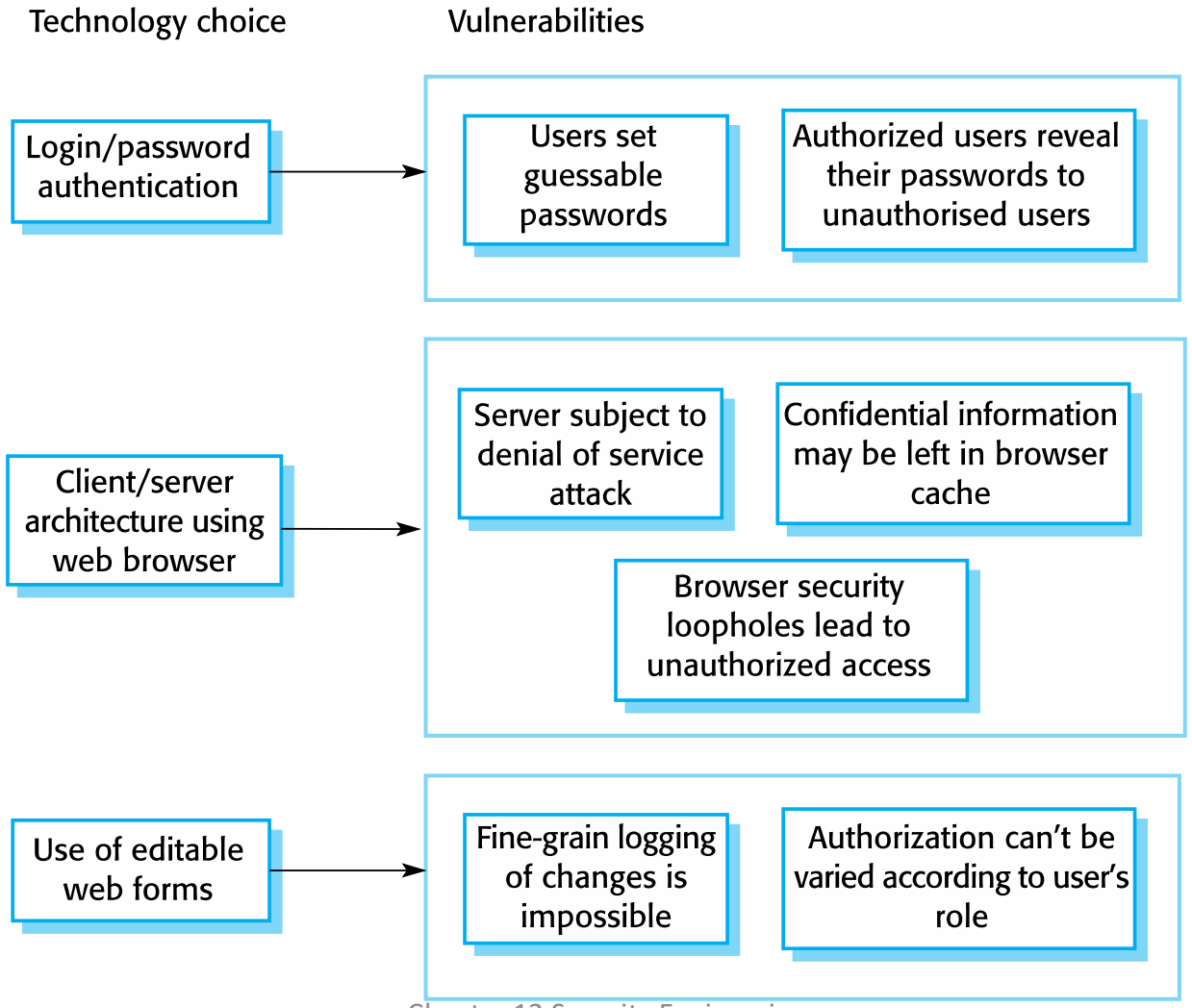
Mentcare misuse case: Intercept transfer

Mentcare system: Intercept transfer (Misuse case)	
Actors	Medical receptionist, Patient records system (PRS), Attacker
Description	A receptionist transfers data from his or her PC to the Mentcare system on the server. An attacker intercepts the data transfer and takes a copy of that data.
Data (assets)	Patient's personal information, treatment summary
Attacks	A network monitor is added to the system and packets from the receptionist to the server are intercepted. A spoof server is set up between the receptionist and the database server so that receptionist believes they are interacting with the real system.

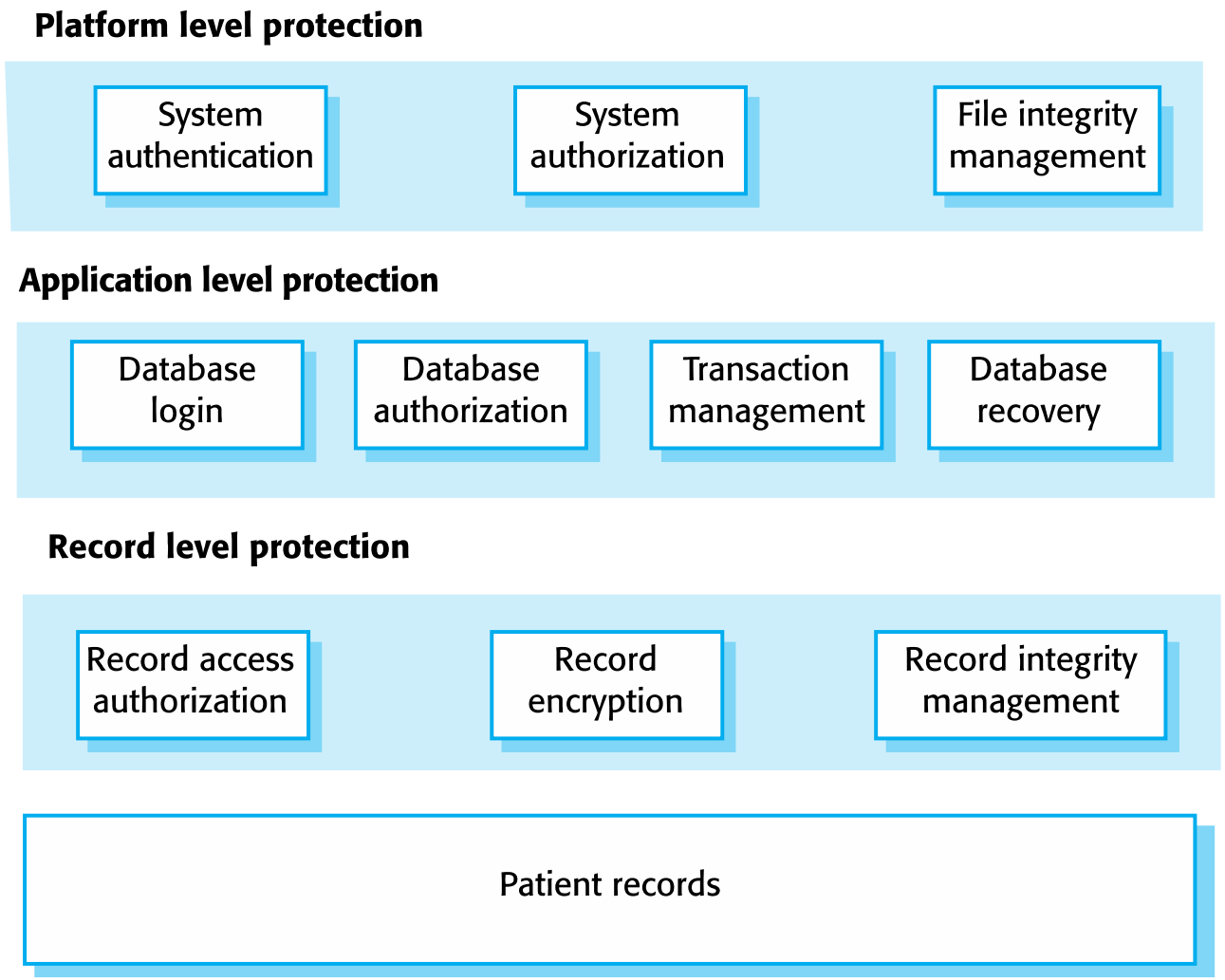
Misuse case: Intercept transfer

Mentcare system: Intercept transfer (Misuse case)	
Mitigations	All networking equipment must be maintained in a locked room. Engineers accessing the equipment must be accredited. All data transfers between the client and server must be encrypted. Certificate-based client-server communication must be used
Requirements	All communications between the client and the server must use the Secure Socket Layer (SSL). The https protocol uses certificate based authentication and encryption.

Vulnerabilities associated with technology choices



A layered protection architecture



Design guidelines for security engineering

- Design guidelines encapsulate good practice in secure systems design
- Design guidelines serve two purposes:
  - They raise awareness of security issues in a software engineering team. Security is considered when design decisions are made.
  - They can be used as the basis of a review checklist that is applied during the system validation process.
- Design guidelines here are applicable during software specification and design

Design guidelines for secure systems engineering

Security guidelines	
Base security decisions on an explicit security policy	
Avoid a single point of failure	
Fail securely	
Balance security and usability	
Log user actions	
Use redundancy and diversity to reduce risk	
Specify the format of all system inputs	
Compartmentalize your assets	
Design for deployment	
Design for recoverability	

Dependable programming guidelines

Dependable programming guidelines

1. **Limit the visibility of information in a program**
2. **Check all inputs for validity**
3. **Provide a handler for all exceptions**
4. **Minimize the use of error-prone constructs**
5. **Provide restart capabilities**
6. **Check array bounds**
7. **Include timeouts when calling external components**
8. **Name all constants that represent real-world values**

Security testing and assurance

Security testing

- Testing the extent to which the system can protect itself from external attacks.
- Problems with security testing
  - Security requirements are ‘shall not’ requirements i.e. they specify what should not happen. It is not usually possible to define security requirements as simple constraints that can be checked by the system.
  - The people attacking a system are intelligent and look for vulnerabilities. They can experiment to discover weaknesses and loopholes in the system.

Security validation

- Experience-based testing
  - The system is reviewed and analysed against the types of attack that are known to the validation team.
- Penetration testing
  - A team is established whose goal is to breach the security of the system by simulating attacks on the system.
- Tool-based analysis
  - Various security tools such as password checkers are used to analyse the system in operation.
- Formal verification
  - The system is verified against a formal security specification.

Examples of entries in a security checklist

Security checklist

1. Do all files that are created in the application have appropriate access permissions? The wrong access permissions may lead to these files being accessed by unauthorized users.

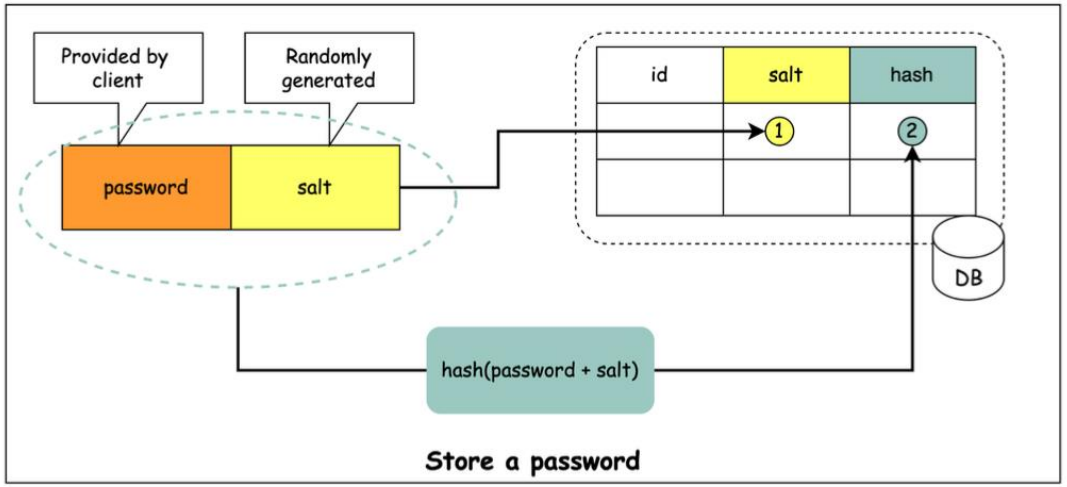
2. Does the system automatically terminate user sessions after a period of inactivity? Sessions that are left active may allow unauthorized access through an unattended computer.

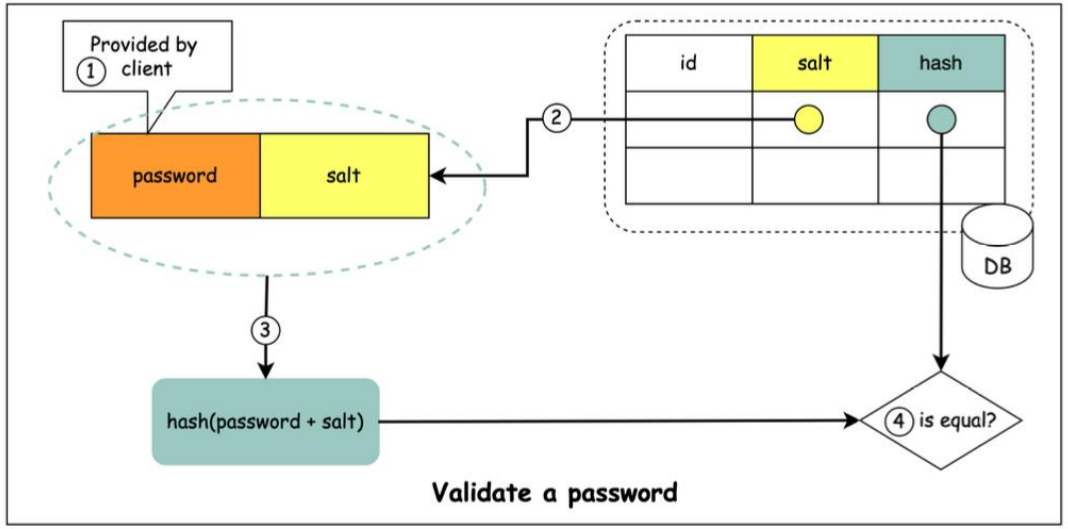
3. If the system is written in a programming language without array bound checking, are there situations where buffer overflow may be exploited? Buffer overflow may allow attackers to send code strings to the system and then execute them.

4. If passwords are set, does the system check that passwords are ‘strong’? Strong passwords consist of mixed letters, numbers, and punctuation, and are not normal dictionary entries. They are more difficult to break than simple passwords.

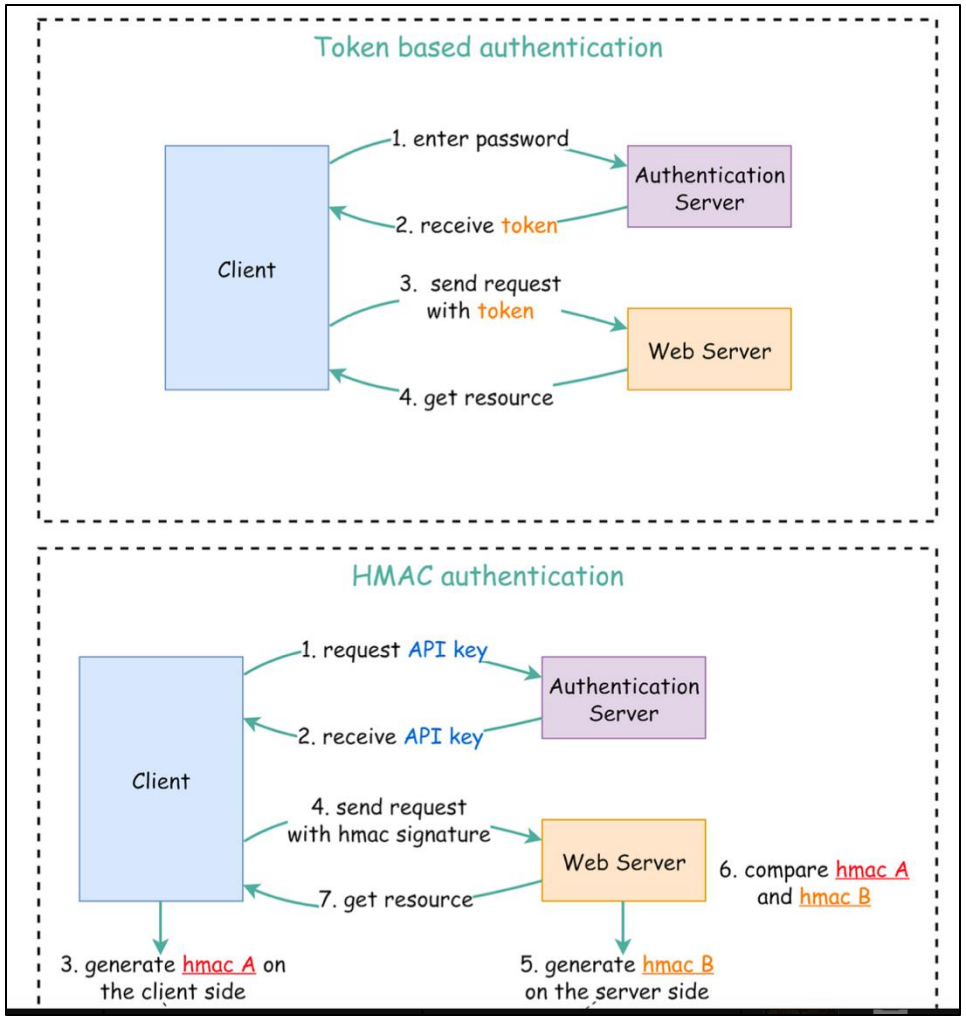
5. Are inputs from the system’s environment always checked against an input specification? Incorrect processing of badly formed inputs is a common cause of security vulnerabilities.

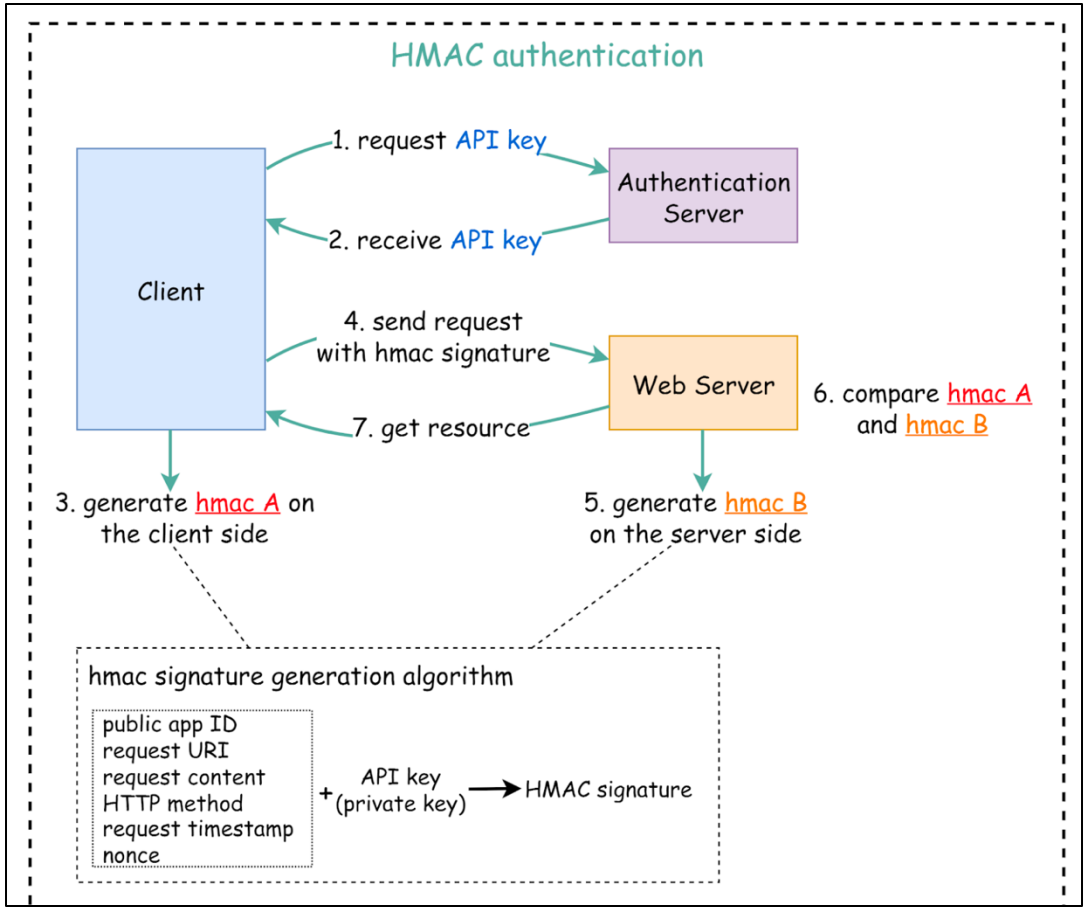
How to store passwords in DB?





How to Design Secure Web API?





END

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Thank You…
Terima kasih..